

Credit Card Approval Prediction using Machine Learning

Project Description:

Banks and financial institutions receive thousands of credit card applications every day. A significant portion of these are rejected due to factors such as high existing loan balances, insufficient income levels, or excessive credit inquiries. Manually reviewing each application is both time-consuming and highly prone to human error, making it an inefficient process at scale.

This project automates the credit card approval decision using machine learning. By training a predictive model on historical applicant data, the system evaluates financial and demographic inputs to determine whether an applicant is likely to be approved or rejected - just as real banks do. Four classification algorithms are applied: Logistic Regression, Random Forest, XGBoost (Gradient Boosting), and Decision Tree.

The best-performing model is saved and integrated into a Flask web application. The project also includes an IBM Watson Machine Learning deployment pipeline, enabling the solution to be hosted on the cloud for scalable, real-time credit card approval predictions accessible through an intuitive user interface.

Scenarios

Scenario 1: Automated Credit Card Application Screening - A bank credit analyst enters a new applicant's financial profile including income type, employment duration, and credit history into the web application. The model returns an instant approval or rejection prediction, enabling the analyst to prioritize applications that require further review.

Scenario 2: High-Risk Applicant Identification and Compliance Review - A financial compliance officer uses the platform to batch-screen applicants with past-due loan records. The feature engineering pipeline converts multi-class payment status codes into binary labels, allowing the model to clearly classify high-risk applicants as ineligible for a new credit card.

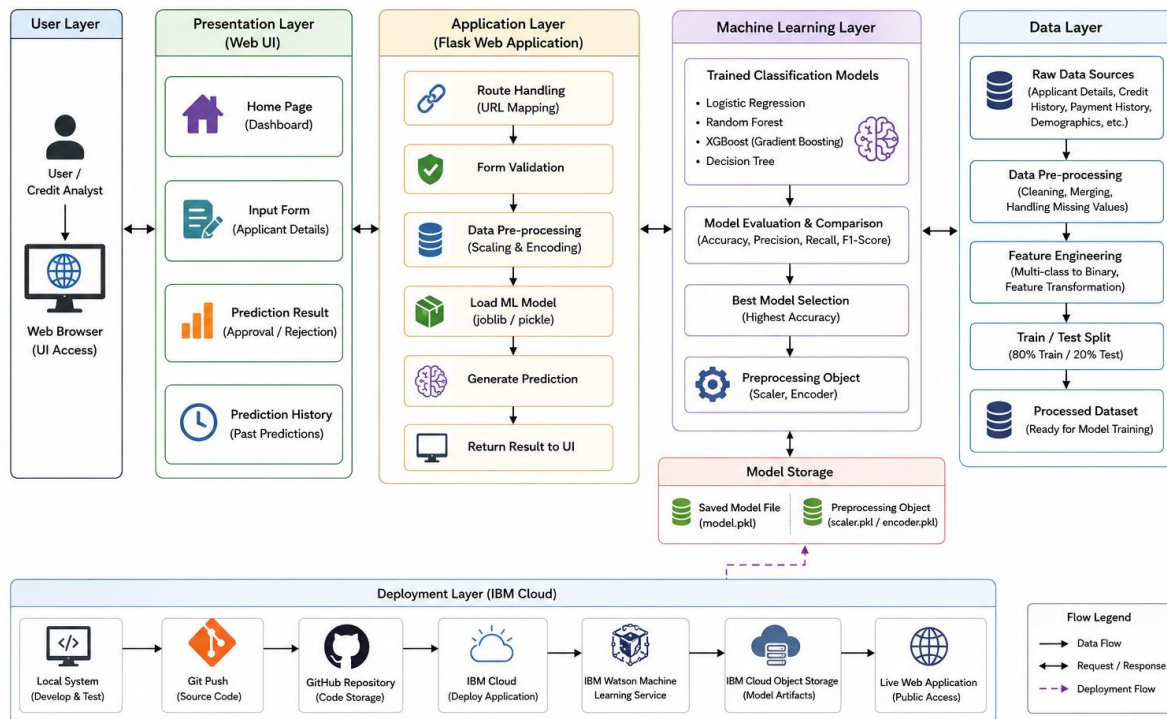
Scenario 3: Customer Self-Service Credit Card Eligibility Check - A prospective customer uses the web application to enter personal and financial details such as income level, employment status, and credit history before submitting a formal credit card application. The system instantly predicts the likelihood of approval, helping the customer understand their eligibility and reducing unnecessary application rejections.

Skills Required

Xgboost, Machine Learning Algorithms, Artificial Intelligence, Decision Tree Learning, NumPy (Python Package), Python (Programming Language), Scikit-Learn (Python Package), Matplotlib (Python Package)

Technical Architecture:

Technical Architecture – Credit Card Approval Prediction Using IBM Watson Machine Learning



Description: The system follows a modular architecture. Historical applicant and credit-history data is cleaned, feature-engineered, and used to train and compare four classification models (Logistic Regression, Random Forest, XGBoost, Decision Tree). The best-performing model is serialized and served through a Flask backend, which takes applicant details submitted via a web form and returns an Approved/Rejected prediction. The application is deployed via an IBM Watson Machine Learning pipeline for scalable, cloud-hosted access.

(Note: If you are using database in your project definitely you need to add ER Diagram along with description)

System Requirements:

Hardware Requirements

Processor: Intel i3 or above

RAM: Minimum 4 GB, recommended 8 GB

Storage: Minimum 2 GB free disk space

Internet Connection: Required for dataset download, package installation, and cloud deployment

Display: Standard monitor with web browser support

Software Requirements

Operating System: Windows / Linux / macOS

Python Version: Python 3.8 or above

Anaconda Navigator: For managing Python environment and Jupyter Notebook

Jupyter Notebook: For data analysis, visualization, preprocessing, and model training

VS Code / PyCharm: For writing Flask application code

Web Browser: Chrome / Edge / Firefox for testing the web application

Project Workflow:

The Credit Card Approval Prediction project follows a structured machine learning workflow to predict whether a credit card application should be approved or rejected. The workflow begins with environment setup, dataset collection, data analysis, preprocessing, feature engineering, model training, model comparison, and final deployment.

Epic 1: Data Collection

Story 1: Download the credit card application dataset (application_record.csv and credit_record.csv) and prepare it for analysis and machine learning model development.

Epic 2: Visualizing and Analysing the Data

Story 1: Import the required Python libraries for data analysis, visualization, and machine learning.

Story 2: Read and explore the dataset to understand its structure, features, and target variable.

Story 3: Perform univariate analysis to examine the distribution of individual variables.

Story 4: Conduct multivariate analysis to identify relationships among multiple applicant and credit factors.

Story 5: Perform descriptive statistical analysis to summarize key characteristics and trends in the dataset.

Epic 3: Data Pre-Processing

Story 1: Identify and remove duplicate records to improve data quality.

Story 2: Detect and handle missing values to ensure dataset completeness.

Story 3: Perform data cleaning and merging operations to create a consistent dataset.

Story 4: Apply feature engineering techniques to create meaningful features for model training.

Story 5: Convert categorical variables into numerical representations suitable for machine learning algorithms.

Epic 4: Model Building

Story 1: Train a Logistic Regression model and evaluate its credit approval prediction performance.

Story 2: Build and test a Random Forest model to improve classification accuracy.

Story 3: Develop a Decision Tree model and analyze its effectiveness in predicting credit card approval.

Story 4: Compare the performance of all trained models using suitable evaluation metrics and select the best-performing model for deployment.

Epic 5: Application Building

Story 1: Design and develop HTML pages to provide a simple and user-friendly interface.

Story 2: Build the Python (Flask) application and integrate the trained credit card approval prediction model.

Story 3: Run, test, and validate the application to ensure accurate credit card approval predictions

and smooth functionality.

Milestone 1: Project Setup & Data Collection

Milestone 1 focuses on setting up the project environment and collecting the dataset required for building the Credit Card Approval Prediction system. The objective is to prepare the development environment and gather reliable data for training the machine learning model. Activity 1.1: Generate a Groq API Key

Activity 1.1: Project Environment Setup

- Installed Python and required libraries.
- Installed Jupyter Notebook for model development.
- Installed Flask for web application development.
- Installed Git for version control.
- Created the project folder structure.

Required Python libraries:

- pandas
- numpy
- scikit-learn
- matplotlib
- seaborn
- flask
- joblib

Activity 1.2: Dataset Collection

The project uses two datasets:

- application_record.csv
- credit_record.csv

The datasets contain applicant information such as:

- Gender
- Income
- Education
- Occupation
- Family Status
- Housing Type
- Number of Children
- Employment Details
- Credit Status

These datasets are merged and cleaned before training the machine learning model.

Milestone 2: Data Preprocessing & Machine Learning Model

Overview

Milestone 2 focuses on preparing the collected data for machine learning. Data preprocessing improves data quality and enables accurate prediction by removing inconsistencies and transforming categorical information into numerical values.

Activity 2.1: Data Preprocessing

Performed the following preprocessing steps:

- Removed duplicate records.
- Removed the **ID** column.
- Merged application and credit datasets.
- Handled missing values.
- Converted categorical values using Label Encoding.
- Created the **Family_Dependency** feature.
- Converted DAYS_BIRTH and DAYS_EMPLOYED into positive values.
- Selected input features and target variable.

Target Variable:

- STATUS

Input Features:

- CODE_GENDER
- FLAG_OWN_CAR
- FLAG_OWN_REALTY

- CNT_CHILDREN
- AMT_INCOME_TOTAL
- NAME_INCOME_TYPE
- NAME_EDUCATION_TYPE
- NAME_FAMILY_STATUS
- NAME_HOUSING_TYPE
- DAYS_BIRTH
- DAYS_EMPLOYED
- FLAG_WORK_PHONE
- FLAG_PHONE
- FLAG_EMAIL
- OCCUPATION_TYPE
- CNT_FAM_MEMBERS
- Family_Dependency

Activity 2.2: Model Training

The dataset was divided into training and testing datasets.

The Random Forest Classifier algorithm was selected because it provides high prediction accuracy and performs well on classification problems.

Steps performed:

- Split dataset into training and testing sets.
- Trained the Random Forest model.
- Evaluated model accuracy.
- Saved the trained model using Joblib.

Generated file:

```
credit_card_model.pkl
```

Milestone 3: Flask Backend Development

Overview

Milestone 3 focuses on integrating the trained machine learning model into a Flask web application to provide real-time credit card approval predictions.

Activity 3.1: Backend Development

Implemented the following functionalities:

- Created Flask application (`app.py`).
- Loaded the trained Random Forest model.
- Developed the Home route (`/`).

```
# Home page
@app.route("/")
def home():
    return render_template("index.html")
```

- Developed the Prediction route (/predict).
- Collected 17 input features from the HTML form.
- Converted the inputs into a NumPy array.
- Passed the inputs to the trained model.
- Displayed the prediction result.

```
app.py
19
20 # Prediction
21 @app.route("/predict", methods=["POST"])
22 def predict():
23
24     try:
25         # Get values from the form
26         features = [
27             float(request.form["CODE_GENDER"]),
28             float(request.form["FLAG_OWN_CAR"]),
29             float(request.form["FLAG_OWN_REALTY"]),
30             float(request.form["CNT_CHILDREN"]),
31             float(request.form["AMT_INCOME_TOTAL"]),
32             float(request.form["NAME_INCOME_TYPE"]),
33             float(request.form["NAME_EDUCATION_TYPE"]),
34             float(request.form["NAME_FAMILY_STATUS"]),
35             float(request.form["NAME_HOUSING_TYPE"]),
36             float(request.form["DAYS_BIRTH"]),
37             float(request.form["DAYS_EMPLOYED"]),
38             float(request.form["FLAG_WORK_PHONE"]),
39             float(request.form["FLAG_PHONE"]),
40             float(request.form["FLAG_EMAIL"]),
41             float(request.form["OCCUPATION_TYPE"]),
42             float(request.form["CNT_FAM_MEMBERS"]),
43             float(request.form["Family_Dependency"])
44         ]
45
46         ## Convert to numpy array
47         final_input = np.array([features])
48
```

Ln 1, Col 1 Spaces: 4 UTF-8 CRLF {} Python

```
22 def predict():
23     try:
24         # Prediction
25         prediction = model.predict(final_input)
26
27         if prediction[0] == 0:
28             result = "✅ Credit Card Approved"
29         else:
30             result = "❌ Credit Card Rejected"
31
32         return render_template("result.html", prediction_text=result)
33
34     except Exception as e:
35         return f"Error: {e}"
36
37 # Run Flask
38 if __name__ == "__main__":
39     app.run(debug=True)
```

Activity 3.2: Prediction Process

The prediction process performs the following steps:

1. User enters applicant information.
2. Flask receives the form data.
3. Data is converted into numerical format.
4. The trained model predicts the result.
5. The application displays either:
 - **Credit Card Approved**
 - **Credit Card Rejected**

Milestone 4: Frontend Development

Overview

Milestone 4 focuses on developing a user-friendly web interface for the Credit Card Approval Prediction system. The frontend was built using HTML, CSS, and Bootstrap to collect applicant details and display prediction results in an easy-to-understand format.

Activity 4.1: Designing the User Interface

The frontend of the application was developed using **HTML5**, **CSS3**, and **Bootstrap 5**.

The following components were implemented:

- Responsive web layout
- Applicant information input form
- Dropdown menus for categorical attributes
- Number input fields for numerical attributes
- Predict button
- Result display page

The application allows users to enter all the required information needed for prediction.

Input Fields

- Gender
- Own Car
- Own Realty
- Number of Children
- Annual Income
- Income Type
- Education Type
- Family Status
- Housing Type
- Days Birth
- Days Employed
- Work Phone
- Phone
- Email
- Occupation Type
- Family Members
- Family Dependency

Credit Card Approval Prediction

Gender Female	Own Car No	Own Realty No
Children 1	Annual Income 2350000	Income Type Commercial Associate
Education Academic Degree	Family Status Civil Marriage	Housing Type Co-op Apartment
Days Birth 90000	Days Employed 25000	Work Phone No
Phone No	Email No	Occupation Type Accountants
Family Members 1	Family Dependency 1	

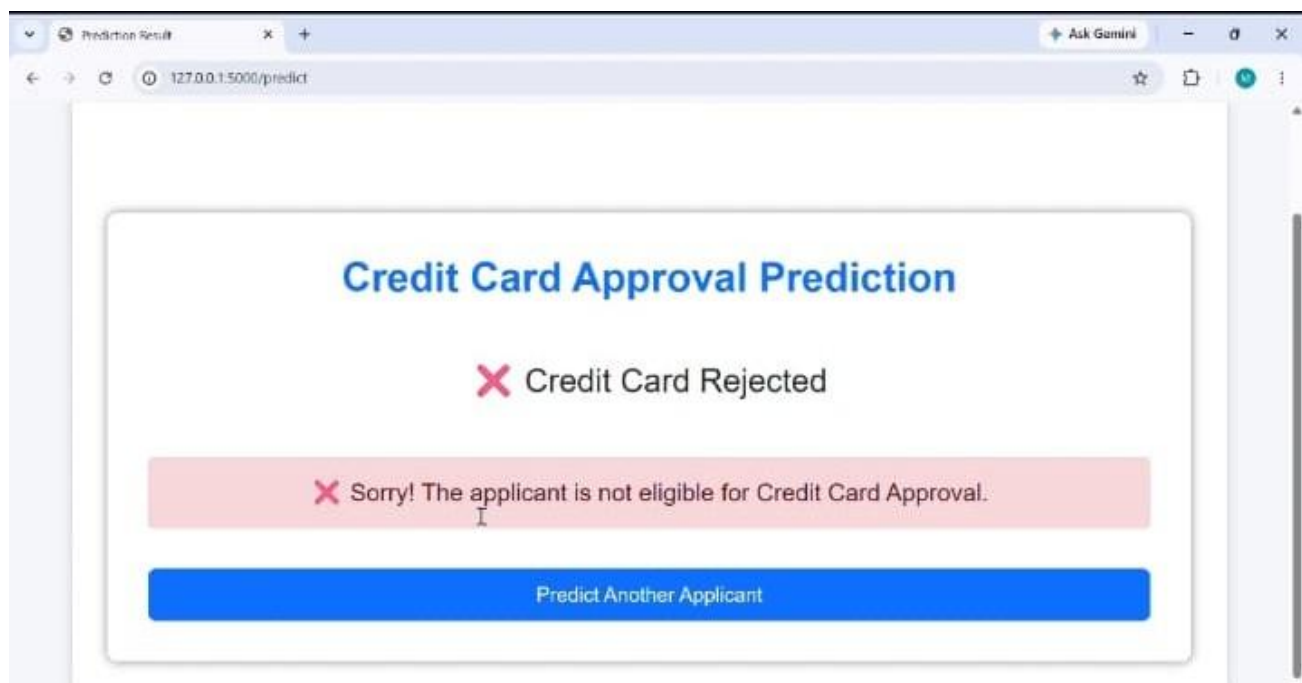
Activity 4.2: Prediction Result Page

After clicking the **Predict** button, the entered information is sent to the Flask backend.

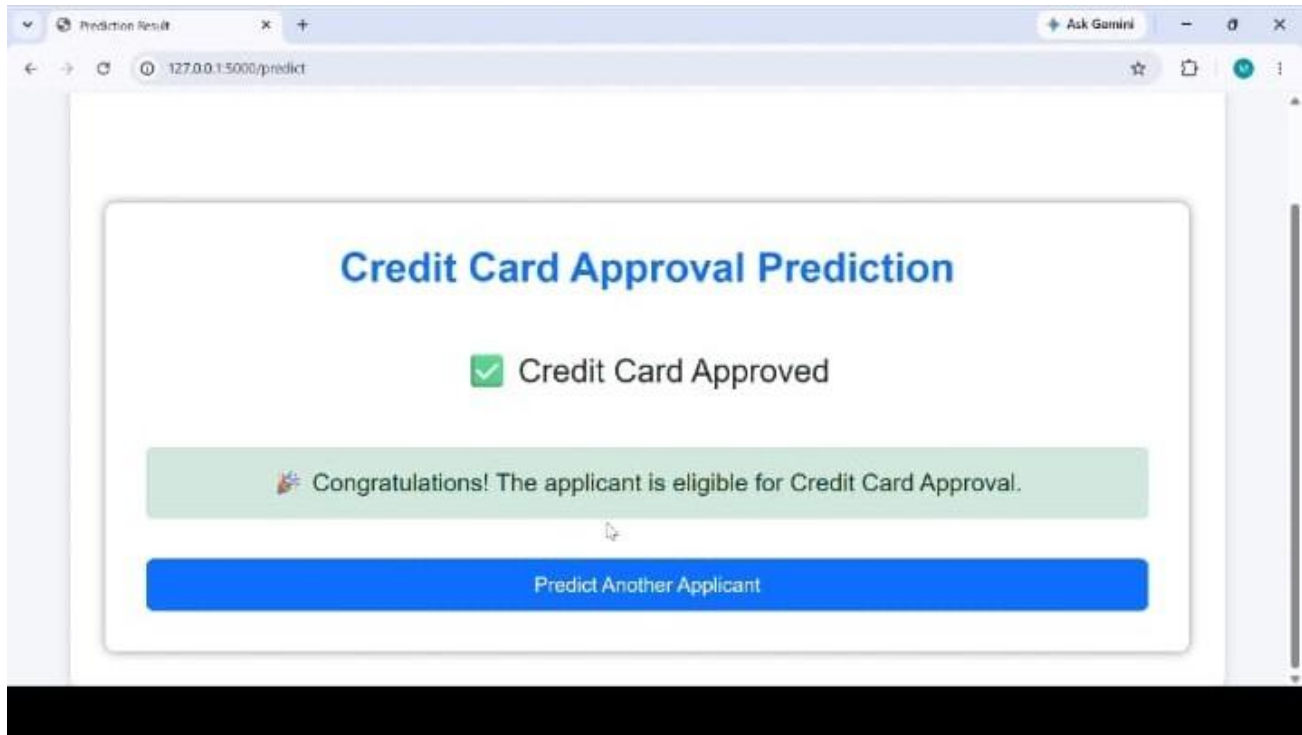
The trained Random Forest model processes the applicant details and returns one of the following results:

- ✓ Credit Card Approved
- ✗ Credit Card Rejected

The result is displayed on a separate webpage with a clear and user-friendly interface.



The screenshot shows a web browser window with the URL `127.0.0.1:5000/predict`. The page displays the "Credit Card Approval Prediction" result. A large red "X" icon is followed by the text "Credit Card Rejected". Below this, a pink message box states: "Sorry! The applicant is not eligible for Credit Card Approval." At the bottom, there is a blue button labeled "Predict Another Applicant".



Frontend Features

- Responsive design using Bootstrap
- Easy-to-use applicant form
- Simple navigation
- Clean layout
- Instant prediction result display
- Input validation for required fields

Milestone 5: Deployment

Overview

Milestone 5 focuses on deploying the Credit Card Approval Prediction application so that it can be accessed through a web browser. The application was first tested locally and then prepared for deployment using GitHub and Render.

Activity 5.1: Preparing the Application

The following steps were completed before deployment:

- Organized the project files into proper folders.
- Created the **templates** folder.
- Created the **static** folder.

- Saved the trained model as **credit_card_model.pkl**.
- Created **requirements.txt**.
- Created **runtime.txt**.
- Verified that all project dependencies were installed.

Required command:

```
pip install -r requirements.txt
```

Activity 5.2: Local Testing

The Flask application was tested locally before deployment.

Run the application using:

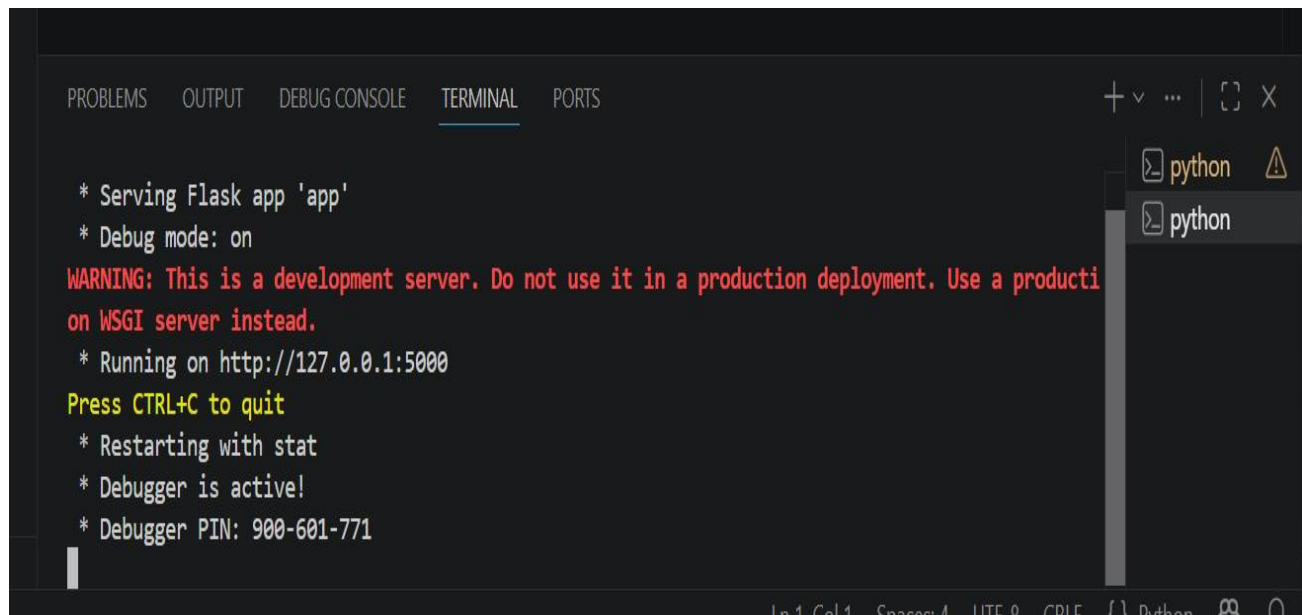
```
python app.py
```

Open the application in a browser:

```
http://127.0.0.1:5000
```

The following tests were performed:

- Verified the Home page.
- Entered sample applicant details.
- Checked prediction accuracy.
- Verified the Result page.
- Confirmed the application worked without runtime errors.



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 900-601-771
```

Activity 5.3: GitHub Repository

The complete project was uploaded to GitHub.

The repository contains:

- Source code
- HTML templates
- CSS files
- Jupyter Notebook
- Machine Learning model
- Dataset files
- requirements.txt
- runtime.txt

GitHub Repository:

```
https://github.com/MahalakshmiReddv18/CreditCardApprovalApp
```

Activity 5.4: Deployment Using Render

The project was deployed using the Render cloud platform.

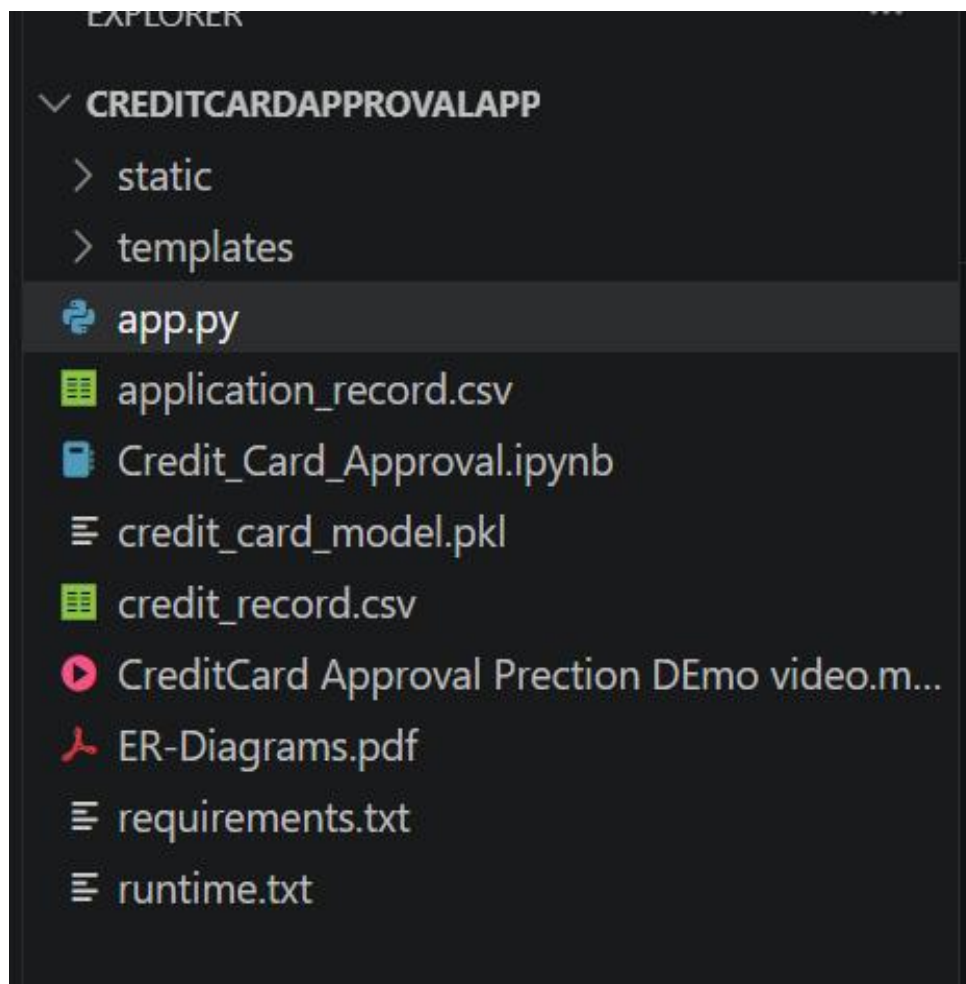
Deployment steps:

1. Upload the project to GitHub.
2. Connect the GitHub repository to Render.
3. Configure the Web Service.
4. Install dependencies using `requirements.txt`.
5. Start the application using:

```
gunicorn app:app
```

6. Render automatically generates a public URL that can be accessed from any device with an internet connection.

Project Structure:



Conclusion

The Credit Card Approval Prediction system was successfully developed and deployed using **Python, Flask, HTML, CSS, Bootstrap**, and the **Random Forest** machine learning algorithm. The application predicts whether a credit card application is likely to be approved or rejected based on applicant information entered through a web interface.

The project includes data preprocessing, machine learning model training, frontend development, backend integration, version control using GitHub, and cloud deployment using Render. Future improvements may include database integration, user authentication, enhanced prediction accuracy using larger datasets, and support for additional machine learning algorithms.